

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Реализация уровня приложения с возможностью многопользовательского доступа к
созданной БД»

Выполнили:

Матвеев Илья Александрович, студент группы N3351

(подпись)

Проверил:

Карманова Наталия Андреевна, ассистент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Ход работы	4
1.1 Создания сервера для базы данных	4
1.2 Настройка авторизации	4
1.3 Идея приложения	7
1.4 Функционал приложения	7
1.5 Инструкция для пользователя и данные для тестов	9
1.6 Исходный код	12
1.7 Тестирование приложения	12
Заключение	15

ВВЕДЕНИЕ

Цель работы – получение навыков использования серверных языков программирования, фреймворков по работе с БД, ORM-систем

Для достижения поставленной цели необходимо решить следующие задачи:

- Реализовать сервер, на котором будет запущена разработанная база данных;
- Настроить политику разграничения доступа;
- Реализовать приложение, которое предоставляет пользователю доступ к просмотру разрешенных ему таблиц.

1 ХОД РАБОТЫ

В данном разделе представлены шаги по выполнению задач, представленных во введении.

1.1 Создания сервера для базы данных

Для того, чтобы реализовать многопользовательский доступ к базе данных. Для этого воспользуемся веб-ресурсом Neon, который предоставляет сервера для postgresql баз данных. Ниже описаны шаги создания сервера и подключения к нему:

- Создание своего приложения и получение к нему пароля – для последующего подключения к серверу;
- Получение хоста, порта и юзернейма для подключения в графическом интерфейсе pgAdmin4;
- Создание в pgAdmin4 сервера с параметрами, полученными ранее;
- Restore базы данных из .backup файла без моделей.

После выполнения данных шагов получаем доступ к базе данных, размещенной на сервере.

1.2 Настройка авторизации

Для начала придется реализовать чистую схему регистрации новых пользователей и авторизации для старых. Воспользуемся классической схемой для postgres баз данных. Сначала нам необходимо создать в базе данных таблицы, которые будут хранить пользователей и текущие сессии. Пароль пользователя будет содержаться в базе в хэшированном виде, для этого воспользуемся расширением pgcrypto. В листинге 1 приведен код, создающий заданные таблицы.

Листинг 1 – Создание таблиц

```
create extension if not exists pgcrypto;

create table if not exists users (
    id          uuid primary key default gen_random_uuid(),
    email       text unique not null,
    password_hash text not null,
    role        text not null default 'student',
    created_at  timestamp default now()
);

create table if not exists sessions (
    token       uuid primary key default gen_random_uuid(),
    user_id     uuid not null references users(id) on delete cascade,
```

```

        created_at    timestamp default now(),
        expires_at    timestamp default (now() + interval '30 days')
    );

```

Далее необходимо написать функцию, которая будет вызываться консольным приложением и моделировать регистрацию и авторизацию пользователей. Работа функции чрезвычайно проста – функция получает параметры (электронная почта, пароль и роль пользователя), хэширует пароль, и записывает эти данные в таблицу пользователей. Код создания функции приведен в листинге 2.

Листинг 2 – Создание функции регистрации

```

create or replace function signup(p_email text, p_password text, p_role text
default 'student')
returns uuid
language plpgsql
as $$
declare
    new_id uuid;
begin
    insert into users(email, password_hash, role)
    values (
        p_email,
        crypt(p_password, gen_salt('bf')),
        p_role
    )
    returning id into new_id;

    return new_id;
end;
$$;

```

Далее необходимо создать такую же понятную функцию авторизации – она получает электронную почту и пароль, сравнивает со значением в базе данных и на основании сравнения возвращает токен, по которому будет идентифицирован текущий пользователь. Однако необходимо защитить схему авторизации базовыми методами – добавить защиту от полного перебора и добавить ошибку о том, что введен неверный логин или пароль. Для этого отдельно создадим таблицу с логинами, которые не прошли проверку, и в функции будем проверять, были ли для таких логинов 5+ неудачных попыток в течение последних минут. Если да – жестко прерываем авторизацию. Код создания таблицы и функции представлен в листинге 3.

Листинг 3 – Схема авторизации

```

create table if not exists failed_logins (
    id          bigserial primary key,
    email       text not null,
    attempted_at timestamp default now()
);
create or replace function login(p_email text, p_password text)
returns uuid

```

```

language plpgsql
as $$
declare
    u_id uuid;
    tkn uuid;
    attempts int;
begin
    select count(*) into attempts
    from failed_logins
    where email = p_email
      and attempted_at > now() - interval '2 minutes';

    if attempts >= 5 then
        raise exception using
            message = 'Слишком много попыток входа. Попробуйте позже.',
            errcode = '28000';
    end if;

    select id into u_id
    from users
    where email = p_email;

    if u_id is null then
        insert into failed_logins(email) values (p_email);
        raise exception using
            message = 'Неверный email или пароль',
            errcode = '28000';
    end if;

    if not exists (
        select 1
        from users
        where id = u_id
          and password_hash = crypt(p_password, password_hash)
    ) then
        insert into failed_logins(email) values (p_email);
        raise exception using
            message = 'Неверный email или пароль',
            errcode = '28000';
    end if;

    delete from failed_logins
    where email = p_email;

    insert into sessions(user_id)
    values (u_id)
    returning token into tkn;

    return tkn;
end;
$$;

```

Далее продолжим улучшать модель авторизации. Создадим корректные роли под представления, при этом не используя LOGIN, потому что авторизация будет идти через клиент. Для начала создадим таблицу, которая свяжет пользователя с его профилем. Код создания таблицы представлен в листинге 4.

Листинг 4 – Создание таблицы профилей пользователей

```
CREATE TABLE app.user_profiles (
    user_id uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    student_id int REFERENCES Students(StudentID),
    teacher_id int REFERENCES Teachers(TeacherID),
    mentor_id int REFERENCES Mentors(MentorID),
    UNIQUE(student_id),
    UNIQUE(teacher_id),
    UNIQUE(mentor_id)
);
```

После этого немного изменим каждое из представлений, чтобы они ссылались на таблицу профилей пользователей.

И в конце добавим сами профили пользователей:

```
CREATE ROLE student NOLOGIN;
CREATE ROLE teacher NOLOGIN;
CREATE ROLE mentor NOLOGIN;
CREATE ROLE admin_my NOLOGIN;
```

На этом реализацию внутренней модели разграничения доступа и регистрации/авторизации можно считать успешно завершённой.

1.3 Идея приложения

Для того, чтобы реально можно было взаимодействовать с базой данных, будем использовать CLI – консольное приложение. Через него пользователи смогут регистрироваться, авторизовываться, получать список доступных таблиц и исполнять запросы, доступные для роли. Кроме этого, будут различные подсказки для пользователя внутри приложения, чтобы он смог разобраться в интерфейсе.

Для создания приложения будет использоваться язык программирования Python в связке с библиотеками psycopg, tabulate, argparse, asyncio – это позволит реализовать консольное приложение на высоком уровне.

Структура приложения следующая:

```
Lab4db/
...config.py - файл конфигурации с параметрами для подключения к БД
...db.py - файл с функциями обращения к БД
...auth.py - файл с функциями авторизации
...main.py - основной файл
```

1.4 Функционал приложения

Основу работы приложения составляет авторизационный механизм, построенный на связке токенов и таблицы сессий. После успешного входа пользователь получает уникальный токен, который сохраняется локально и автоматически подставляется во все

последующие запросы. Это избавляет от необходимости повторного ввода данных и позволяет обеспечить корректную идентификацию пользователя при каждом обращении к серверу. Истечение срока действия токена, удаление сессии или попытки подбора пароля корректно обрабатываются и приводят к отказу в доступе, что повышает общую безопасность системы.

Приложение учитывает ролевую модель доступа, встроенную в структуру базы данных. В зависимости от роли (student, teacher, mentor, admin_my) пользователь получает доступ лишь к тем представлениям и данным, которые соответствуют его полномочиям. Например, студент может видеть только своё расписание, задания и результаты; преподаватель — данные своих курсов и оценивание студентов; ментор — проверки работ; администратор — технические таблицы и возможность удаления записей. Такой подход позволяет обеспечить строгую изоляцию данных и предотвращает несанкционированный доступ.

Важной составляющей функционала является команда select, позволяющая выполнять выборки из представлений, доступных текущей роли. Приложение автоматически показывает разрешённые поля, проверяет корректность вводимых пользователем запросов, предотвращает обращение к недоступным данным и выводит результат в удобном табличном формате. Поддерживается механизм distinct, а также безопасное выполнение SQL-запросов через функцию-обёртку, работающую на стороне сервера.

Для преподавателей и менторов реализован механизм оценивания и проверки работ. Команды grade и check автоматически проверяют, имеет ли пользователь право выставить оценку конкретному студенту или проверять конкретный результат. Такая логика привязана к данным в таблицах курсов, групп и связей преподавателей, что исключает возможность некорректного вмешательства.

Для администратора предусмотрена команда delete, позволяющая безопасно удалять записи из заранее определённого списка таблиц. Перед удалением приложение выводит расшифровку записи (ФИО студента, название курса и т.п.), требует дополнительного подтверждения и только после этого выполняет операцию. Такая двухступенчатая схема снижает вероятность ошибочного удаления ценных данных.

С технической точки зрения приложение использует асинхронный драйвер PostgreSQL и модуль asyncio, что позволяет выполнять запросы параллельно и реагировать на задержки сетевого соединения без блокировки интерфейса. Библиотека argparse

обеспечивает удобный и понятный интерфейс команд, а модуль `tabulate` делает вывод структурированных данных визуально читаемым.

1.5 Инструкция для пользователя и данные для тестов

Данное приложение представляет собой консольный клиент для взаимодействия с учебной базой данных. Все операции выполняются через команды, запускаемые из терминала. Ниже приводится описание всех доступных команд и правила их использования.

1. Регистрация пользователя

Команда:

```
main.exe signup <email> <password> [role]
```

Назначение:

Создаёт нового пользователя в системе. Роль указывается по желанию; если она не указана, используется роль «student».

Пример использования:

```
main.exe signup user@mail.com  
qwerty123 teacher
```

2. Авторизация (вход в систему)

Команда:

```
main.exe login <email> <password>
```

Назначение:

Выполняет вход пользователя и сохраняет выданный сервером токен, который используется во всех последующих запросах.

Пример:

```
main.exe login user@mail.com  
qwerty123
```

3. Просмотр текущего пользователя

Команда:

```
main.exe whoami
```

Назначение:

Отображает информацию о текущем авторизованном пользователе: его идентификатор, электронную почту и роль.

Пример:

```
main.exe whoami
```

4. Выход из аккаунта

Команда:

```
main.exe logout
```

Назначение:

Удаляет локально сохранённый токен и завершает текущую сессию.

Пример:

```
main.exe logout
```

5. Просмотр доступных сущностей базы данных

Команда:

```
main.exe tables
```

Назначение:

Отображает список таблиц и представлений, доступных текущей роли пользователя.

Набор доступных сущностей зависит от привилегий.

Пример:

```
main.exe tables
```

6. Выполнение запросов на выборку данных

Команда:

```
main.exe select [fields]
```

Назначение:

Позволяет выводить данные из представления, соответствующего роли пользователя.

Если поля не указаны, выводится список доступных полей.

Примеры:

– вывод доступных полей:

```
main.exe select
```

– выбор конкретных полей:

```
main.exe select assignment_title, deadline
```

7. Выставление оценки студенту

Команда:

```
main.exe grade <assignment_id> <student_id> --grade <оценка> [--comment  
«текст»]
```

Назначение:

Позволяет преподавателю или ментору выставить оценку конкретному студенту по заданию.

Система автоматически проверяет полномочия пользователя.

Пример:

```
main.exe grade 3 12 --grade 8 --comment "Хорошая работа"
```

8. Проверка работы ментором

Команда:

```
main.exe check <result_id> [--grade <оценка>] [--comment_fr «текст»]
```

Назначение:

Позволяет ментору проверить работу студента, оставить комментарий и при необходимости выставить оценку.

Пример:

```
main.exe check 15 --grade 9 --comment_fr "Отлично выполнено"
```

9. Удаление записей (для администратора)

Команда:

```
main.exe delete <table> <id> [--yes]
```

Назначение:

Удаляет запись из выбранной таблицы. Перед удалением приложение выводит подробную информацию о найденной записи и запрашивает подтверждение.

Флаг «--yes» позволяет выполнить удаление без подтверждения.

Примеры:

– удаление с подтверждением:

```
main.exe delete students 12
```

– удаление без подтверждения:

```
main.exe delete courses 5 --yes
```

10. Общие правила использования

- Все команды выполняются только после авторизации;
- Пользователю доступны только те операции, которые разрешены его ролью;
- Токен сохраняется автоматически после входа и действует до выхода;
- В случае ошибок приложение выводит текстовое сообщение с описанием проблемы;

Данные для тестирования:

– Преподаватели:

```
1 melnikovde@gmail.com Tch-Melnikov-91!  
2 polinashtorm@gmail.com Tch-Shtorm-82!  
3 coolmathematic@gmail.com Tch-Math-73!  
4 alexandrovskayani@gmail.com Tch-Alg-64!  
5 velnik@gmail.com Tch-Velnik-55!  
6 englishteacher@gmail.com Tch-English-46!  
7 ludmilaphys@gmail.com Tch-Phys-37!
```

– Студенты:

```
1 ivanov@example.com Std-01-xA7!  
2 zolinai@mail.ru Std-02-dF9!  
3 kirinnv@gmail.com Std-03-pL3!  
4 пётр.васильев72@yandex.ru Std-04-qR8!  
5 пётр.смирнов65@example.com Std-05-mT2!
```

```

6  иван.сидоров47@example.com  Std-06-sK5!
7  пётр.васильев50@gmail.com  Std-07-bN1!
8  пётр.иванов86@yandex.ru  Std-08-hC4!
9  пётр.сидоров80@mail.ru  Std-09-zV6!
10  никита.иванов52@example.com  Std-10-gW7!
11  андрей.кузнецов84@yandex.ru  Std-11-jE3!
12  дмитрий.кузнецов77@gmail.com  Std-12-uY9!
13  андрей.петров80@example.com  Std-13-rQ1!
  – Менторы:
1  mentor.sergey.kuznetsov@example.com  Mnt-01-sG4!
2  mentor.anna.smirnova@example.com  Mnt-02-vL8!
3  mentor.dmitry.pavlov@example.com  Mnt-03-kD2!
4  mentor.ekaterina.volkova@example.com  Mnt-04-pH7!
5  mentor.maria.sokolova@example.com  Mnt-05-cJ9!
6  mentor.andrey.ilyin@example.com  Mnt-06-wF1!
7  mentor.natalia.grigorieva@example.com  Mnt-07-nB5!
8  mentor.mikhail.zakharov@example.com  Mnt-08-xM3!
9  mentor.olga.kiseleva@example.com  Mnt-09-tP6!
10  mentor.viktor.tikhonov@example.com  Mnt-10-yZ8!
  – Админ:
admin@onlineschool.local
Adm-Root-99!

```

1.6 Исходный код

Исходный код приложения можно просмотреть в приложенном архиве app.zip, в каждом из файлов.

1.7 Тестирование приложения

Для тестирования необходимо проверить все возможности. Ниже, на рисунках 1-5, представлены результаты каждого запроса.

```

(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe whoami
Вы не авторизованы.

(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe -h
usage: main.exe [-h] {select,signup,login,whoami,logout,tables,grade,check,delete} ...

CLI-интерфейс для работы с БД. Поддерживает авторизацию, просмотр данных, оценивание, проверку работ и административные операции.

positional arguments:
  {select,signup,login,whoami,logout,tables,grade,check,delete}
  select                Выборка данных из представления, доступного вашей роли.
  signup               Регистрация нового пользователя.
  login                Вход в аккаунт.
  whoami               Показать текущего авторизованного пользователя.
  logout              Выйти из аккаунта.
  tables               Показать список таблиц и представлений, доступных вашей роли.
  grade               Выставить оценку студенту (для преподавателей и менторов).
  check               Проверить работу как ментор.
  delete              Удаление записи (только для admin_my).

optional arguments:
  -h, --help            show this help message and exit

```

Рисунок 1 – Функции whoami и help

```
(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe login ivanov@example.com Std-01-xA7!
Вход выполнен. Токен сохранён: 517e3d2e-0140-4229-9ae5-94171e264c10

(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe select

Поля, доступные роли 'student' в представлении 'student_view':
```

Поле	Источник
assignment_title	assignments
comment_fr	results
course_name	courses
date	schedule
deadline	assignments
grade	results
group_name	groups
lesson_type	schedule
student_name	students
studentid	students
submissiondate	results
time	schedule

Рисунок 2 – Функции login и select без аргументов

```
(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe delete
usage: main.exe delete [-h] [--yes] table id
main.exe delete: error: the following arguments are required: table, id

(venv) C:\Users\sie45\PycharmProjects\lab4bd\dist>main.exe delete results 2
Удаление доступно только admin_my.
```

Рисунок 3 – Удаление без прав админа

group_name	groups
groupid	groups
student_name	students
studentid	students
submissiondate	results
teacher_name	teachers
teacherid	teachers

```
C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select assignmentid, assignment_title, studentid, student_name, grade
usage: main.exe [-h] {select,signup,login,whoami,logout,tables,grade,check,delete} ...
main.exe: error: unrecognized arguments: assignment_title, studentid, student_name, grade

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select assignmentid,assignment_title,studentid,student_name,grade
```

assignmentid	assignment_title	studentid	student_name	grade
1	Первое задание	2	Золин Алексей Игоревич	67
1	Первое задание	1	Иванов Иван Иванович	
2	Второе задание	2	Золин Алексей Игоревич	
2	Второе задание	1	Иванов Иван Иванович	
3	Проверка	2	Золин Алексей Игоревич	
3	Проверка	1	Иванов Иван Иванович	

```
C:\Users\svirz\Downloads\AyuGram Desktop>main.exe grade 2 2 --grade 5 --comment "Хорошая работа"
Вы не ведёте этот курс или не работаете с этим студентом.
```

Рисунок 4 – Проверка select

```

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select

Поля, доступные роли 'mentor' в представлении 'mentor_view':
+-----+-----+
| Поле           | Источник   |
+-----+-----+
| assignment_title | assignments |
+-----+-----+
| comment_fr       | results    |
+-----+-----+
| deadline         | assignments |
+-----+-----+
| grade           | results    |
+-----+-----+
| mentor_name      | mentors    |
+-----+-----+
| mentorid         | mentors    |
+-----+-----+
| student_name     | students   |
+-----+-----+
| studentid        | students   |
+-----+-----+
| submissiondate   | results    |
+-----+-----+

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select studentid,student_name,assignment_title,grade
+-----+-----+-----+-----+
| studentid | student_name | assignment_title | grade |
+-----+-----+-----+-----+
|          3 | Кирин Никита Владимирович | Первое задание |      85 |
+-----+-----+-----+-----+

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select id,student_name,assignment_title,grade
Недоступные поля: id

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select result_id,student_name,assignment_title,grade
Недоступные поля: result_id

C:\Users\svirz\Downloads\AyuGram Desktop>main.exe select result_id
Недоступные поля: result_id

```

Рисунок 5 – Проверка приложения от ментора

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано консольное приложение, позволяющее пользователям взаимодействовать с системой в соответствии с их ролевыми правами. Приложение поддерживает регистрацию, авторизацию, выборку данных из доступных представлений, оценивание и административные операции. Использование асинхронных библиотек, функций базы данных и структурированной архитектуры позволило создать удобный и расширяемый инструмент для работы с БД.

В результате работы были получены практические навыки разработки серверной логики, освоены методы построения безопасной модели доступа, а также приобретён опыт создания прикладного клиентского интерфейса для взаимодействия с удалённой базой данных. Цель лабораторной работы достигнута - реализован полный цикл: от создания сервера и настройки безопасности до разработки полноценного приложения, обеспечивающего корректный доступ пользователей к данным.